



RAPPORT DE PROJET

Livraison d'organes par drones

Diagrammes de Voronoï & Triangulation de Delaunay

Projet P.G.L. Delaunay / Voronoï

Année 2025–2026

Tuteur Souhila Arib

Équipe

Ahmed Jeddou Mohamed Lemine Abdy Yousef

Boukah Boukah

Table des matières

1	Introduction	2
2	Organisation de l'équipe	2
3	Description du projet	2
3.1	Fonctionnalités principales	2
3.2	Diagramme de classes	3
3.3	Cas d'utilisation	4
4	Algorithmes : Voronoï & Delaunay	4
4.1	Voronoï	4
4.2	Delaunay	4
4.3	Sélection de la base optimale	5
5	Points utilisateurs — choix de conception	5
6	Interface JavaFX	5
7	Problèmes rencontrés & solutions	7
8	Résultats	8
9	Conclusion	8

1 Introduction

Le transport d'organes est un enjeu médical critique où chaque minute compte. Notre projet simule un système de livraison d'organes par drones entre centres de prélèvement et hôpitaux, en utilisant les diagrammes de Voronoï et la triangulation de Delaunay pour optimiser l'affectation des drones et les trajets.

Problématique

Comment optimiser le transport d'organes par drones tout en réduisant les délais, garantissant la sécurité des greffons et en assurant la traçabilité des missions ?

2 Organisation de l'équipe

Trois membres, méthode Agile, Mme Souhila Arib comme tutrice (Product Owner), Abdy Yousef comme Scrum Master. Réunions hebdomadaires via Discord, suivi via GitHub.

Membre	Rôle
Abdy Yousef (Scrum Master)	Classes médicales : <code>MedicalSite</code> , <code>Hospital</code> , <code>CollectionCenter</code> , <code>MedicalStaff</code> , <code>DeliveryRequest</code> , <code>UserPoint</code> , tests CMD
Ahmed Jeddou Mohamed Lemine	Drones & optimisation : <code>Drone</code> , <code>DroneBase</code> , <code>Route</code> , <code>OptimizationService</code> , <code>VoronoiDiagram</code> , <code>DelaunayTriangulation</code>
Boukah Boukah	Carte & interface : <code>MapModel</code> , <code>Mission</code> , <code>Triangle</code> , <code>Main</code> , interface JavaFX complète

3 Description du projet

3.1 Fonctionnalités principales

- Ajout, suppression, déplacement de sites médicaux (hôpitaux, centres) et bases de drones
- Import CSV en masse et export/import binaire de la carte
- Gestion des médecins (`MedicalStaff`) par hôpital — chaque médecin = un point utilisateur Voronoï
- Sélection automatique de la base optimale et du drone disponible
- Animation temps réel du vol avec chronomètre
- Inspection interactive des zones Voronoï et triangles Delaunay
- Zoom + pan, drag & drop des éléments sur la carte

→ Voir le diagramme de classes sur Lucidchart

3.3 Cas d'utilisation

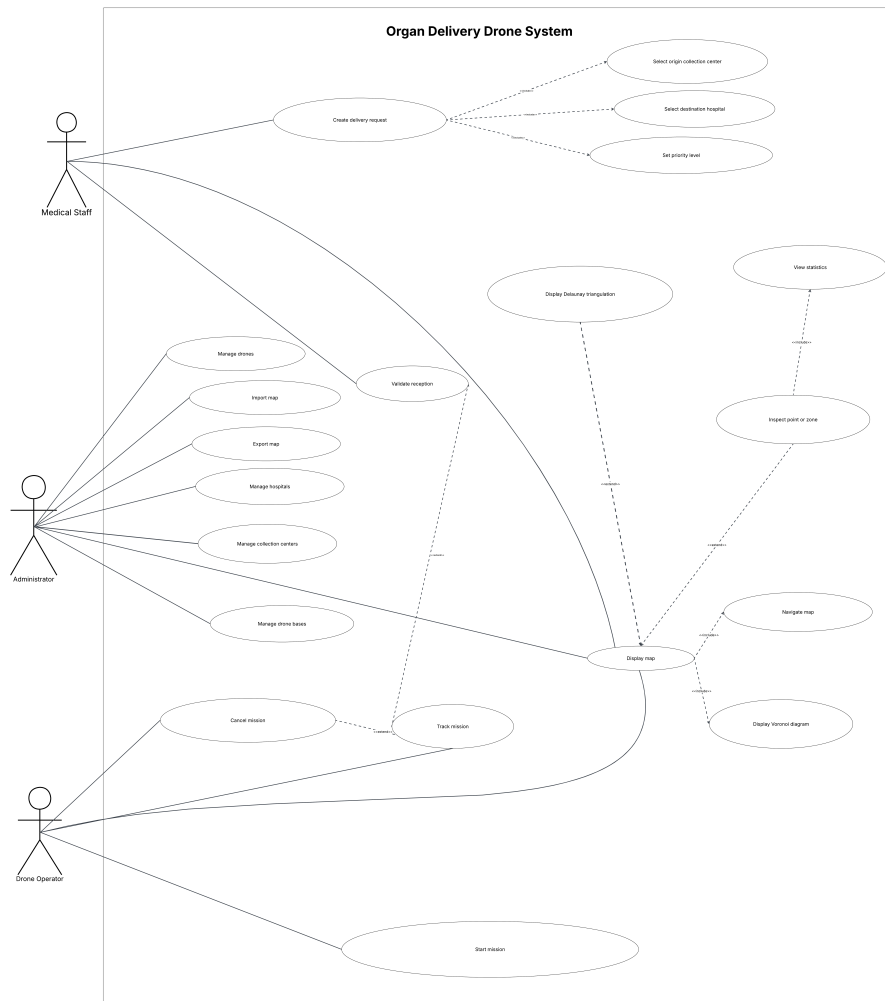


FIGURE 2 – Diagramme de cas d'utilisation

→ [Voir le diagramme de cas d'utilisation sur Lucidchart](#)

4 Algorithmes : Voronoï & Delaunay

4.1 Voronoï

Chaque point de la grille est affecté au site médical le plus proche (balayage par distance euclidienne). Utilisé pour : trouver le centre de prélèvement le plus proche d'un hôpital, calculer les statistiques de zone (surface, densité de médecins, distances).

4.2 Delaunay

Algorithme incrémental avec test du cercle circonscrit. Utilisé pour : valider l'adjacence géométrique entre centre et hôpital (`areDelaunayNeighbours`), afficher le réseau de voisinage, et inspecter les triangles (surface, distances $AB/BC/CA$, médecins par sommet).

4.3 Sélection de la base optimale

$$score(base) = d(base \rightarrow centre) + d(base \rightarrow hôpital)$$

La base avec le score minimal est choisie automatiquement.

5 Points utilisateurs — choix de conception

Adaptation du cahier des charges

Le CDC demande des *points utilisateurs* génériques. Dans notre projet, seuls les **médecins des hôpitaux** peuvent émettre des demandes de livraison. Chaque médecin enregistré (**MedicalStaff**) est automatiquement créé comme **UserPoint** à la position de son hôpital, ce qui l'intègre dans les statistiques Voronoï (nombre de demandeurs par zone, distances).

L'ajout de médecins en masse est réalisé via le bouton *Add Random Hospitals* qui génère des hôpitaux avec un médecin par défaut, ou via *Add Doctor to Hospital* pour un ajout manuel. La suppression d'un médecin supprime automatiquement son UserPoint.

6 Interface JavaFX

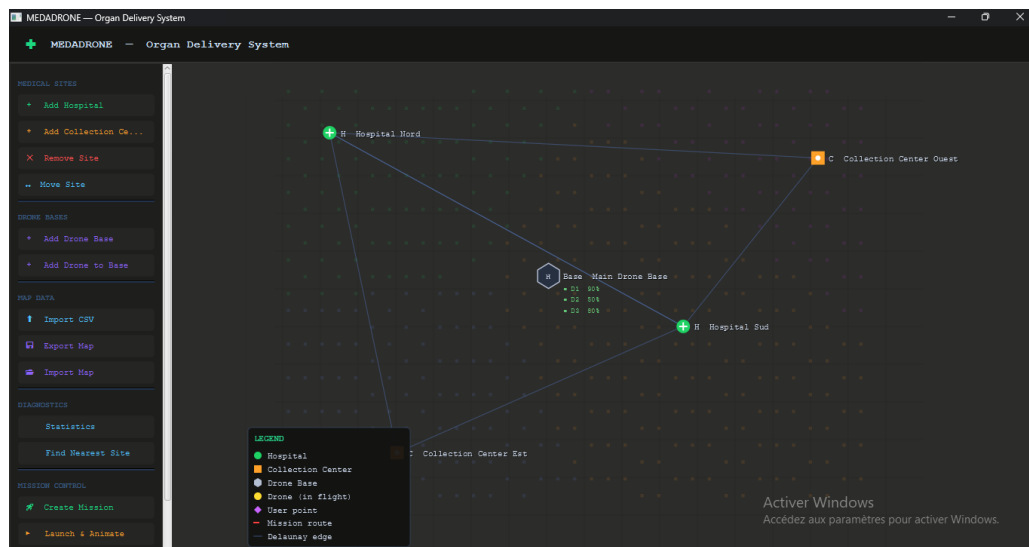


FIGURE 3 – Interface principale

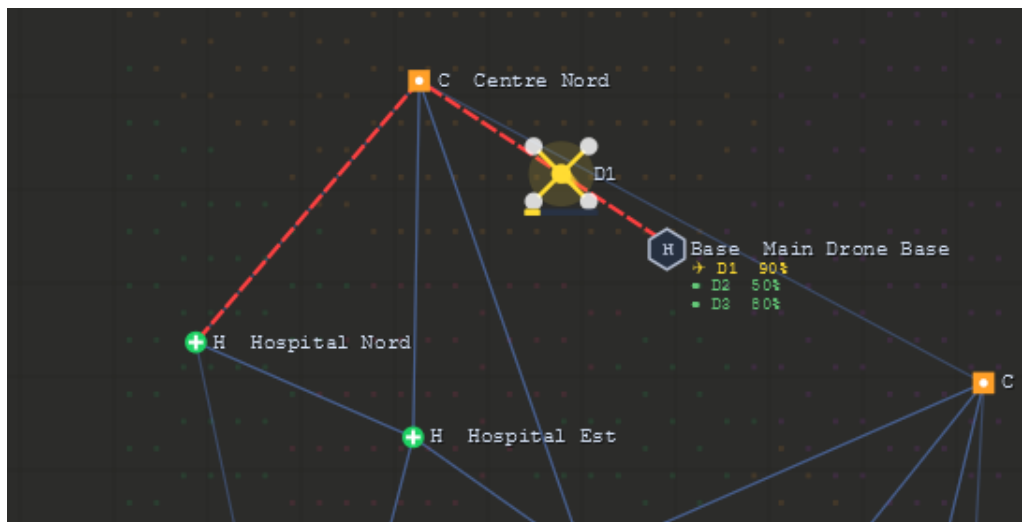


FIGURE 4 – Animation de mission

Thème sombre bleu marine (#0A1428), sidebar structurée en sections, panneaux d'info au clic, panneau stats déplaçable, chronomètre de mission, barre de progression du drone.

7 Problèmes rencontrés & solutions

Problème	Solution
JavaFX ne se lance pas	Ajout de javafx-graphics dans pom.xml et correction de mainClass vers ui.MainApp.
Drone décolle depuis une mauvaise position	Les drones sont initialisés à la position exacte de leur base. findBasePosition() retrouve la base contenant le drone.
Route passant par un hôpital intermédiaire inutile	Suppression du waypoint Delaunay parasite. Route strictement base → centre → hôpital.
OptimizationService sans MapModel	Ajout d'un 2 ^e paramètre MapModel au constructeur.
Barre de fenêtre JavaFX absente	primaryStage.initStyle(StageStyle.DECORATED).
Export carte : accès refusé	Chemin par défaut vers ~/medadrone_map.bin, création automatique des dossiers parents.
Points utilisateurs : concept inadapté	Dans notre contexte, seuls les médecins émettent des demandes. Chaque MedicalStaff est enregistré comme UserPoint à la position de son hôpital. L'ajout/suppression d'un médecin synchronise automatiquement le UserPoint associé.
MedicalStaff sans getId()	Ajout des getters manquants : getId(), getFirstName(), getLastName(), getPhoneNumber().
Méthodes MedicalStaff absentes de MapModel	Ajout de addMedicalStaff(), removeMedicalStaff(), getMedicalStaffByHospital(), getAllMedicalStaff(), findMedicalStaffById() avec liste dédiée medicalStaffList.
11 erreurs <i>unclosed string literal</i>	Les \n dans les chaînes Java étaient de vrais sauts de ligne. Correction de l'échappement.

8 Résultats

Fonctionnalité	Statut
Ajout / suppression / déplacement de sites	OK
Import CSV + export/import binaire	OK
Voronoi affiché et calculable	OK
Delaunay affiché et calculable	OK
Sélection optimale base + drone	OK
Animation drone temps réel + chronomètre	OK
Gestion des médecins (UserPoints)	OK
Inspection zones Voronoi + triangles	OK
Zoom + pan + drag&drop	OK
Version ligne de commande complète	OK
JavaDoc générée dans docs/	OK

9 Conclusion

Ce projet nous a permis de maîtriser la géométrie computationnelle (Voronoi, Delaunay), la programmation orientée objet Java, et le développement d'interfaces JavaFX. La principale adaptation par rapport au cahier des charges générique concerne les points utilisateurs, remplacés par les médecins des hôpitaux dans notre contexte médical.